Q4 Computational Problem: Income Prediction

In this problem, we will fit logistic regression models on the Adult dataset. This is a classical dataset in machine learning where want to predict if an individual in the 1994 US Census earns over $50K annually. Our goal is to give you some practice with essential aspects of model development and evaluation when building a model that outputs risk predictions. We'll use a processed dataset with binary features to speed things up (link available in problem set pdf), and provide most of the code you need in this starter notebook.

Helper Code

Implement L1 Logistic Regression Class's `fit(self, X, y)` function below by modifying the L2 Logistic Regression Class code from Q4. For part (a), you could fit Vanilla Logistic Regression by setting penalty to 0 to the L2 Logistic Regression you implemented in Q4, but you would want to implement this class for part (b) and (c).

In [18]:
```
import matplotlib.pyplot as plt
from tqdm import tqdm
import pandas as pd
%pip install cvxpy
import cvxpy as cp
import numpy as np
from sklearn.metrics import RocCurveDisplay
from sklearn.calibration import CalibrationDisplay
from sklearn.model_selection import StratifiedKFold

#### L1 Logistic Regression via CVX
class L1LogisticRegression(object):

    def __init__(self, C = 0.0):
        assert np.greater_equal(C, 0.0)
        self.C = cp.Parameter(nonneg=True)
        self.C.value = C

    # main methods
    def check_data(self, X, y):
        assert X.ndim == 2
        assert X.shape[0] > 0
        assert np.isfinite(X).all()
        assert np.isfinite(y).all()
        assert np.isin(y, (0, 1)).all()
        return True

    def fit(self, X, y):
        """
        :return: coefficients of optimal model
        """
        assert self.check_data(X, y)
        self.n = X.shape[0]
        self.d = X.shape[1]
        self.X = X
        self.y = y
        self.w = cp.Variable(self.d)
        self.t = cp.Variable(1)
        # YOUR CODE HERE
        J = cp.sum(cp.multiply(self.y, self.X @ self.w + self.t) - cp.logistic(self.X @ self.w + self.t))
        self.problem = cp.Problem(cp.Maximize(J / self.n - self.C * cp.norm1(self.w)))
        self.problem.solve()

    #### Classifier Properties ###
    @property
    def has_solution(self):
        return self.w.value is not None

    @property
    def intercept(self):
        if self.has_solution:
            return self.t.value
        else:
            return np.nan

    @property
    def coefs(self):
        if self.has_solution:
            return self.w.value
        else:
            return np.repeat(np.nan, self.d)

    #### prediction API ####
    def decision_function(self, X):
        """
        returns confidence score
        :param X:
```

```
        :return: vector of confidence scores for each row in X
        """
        scores = X.dot(self.coefs) + self.intercept
        return scores

    def predict_proba(self, X):
        """
        predicts probability of positive class
        :param X: feature matrix with d columns features
        :return: vector of predicted probabilities for each row in X
        """
        scores = self.decision_function(X)
        probs = np.zeros_like(scores)
        pos_idx = np.greater_equal(scores, 0)
        neg_idx = np.logical_not(pos_idx)
        probs[pos_idx] = 1.0 / (1.0 + np.exp(-scores[pos_idx]))
        probs[neg_idx] = np.exp(scores[neg_idx]) / (1.0 + np.exp(scores[neg_idx]))
        return probs

    def predict(self, X):
        """
        predicts label for each row in X
        :param X: feature matrix with d columns features
        :return: vector of confidence scores for each row in X
        """
        probs = self.predict_proba(X)
        yhat = np.greater(probs, 0.5)
        return yhat

#### HELPER FUNCTIONS ####
def compute_log_loss(y_true, y_proba, eps=1e-15):
    """
    computes mean logistic loss using labels and probability predictions
    :param y_true: vector of true labels - y_true[i] in (-1,+1) and y_true[i] (0,1) are both OK
    :param y_proba: vector of predicted probabilities
    :param eps: minimum distance that y_pred must maintain from 0 and 1
    :return: log_loss
    """
    p = np.clip(y_proba, eps, 1.0 - eps)
    y = y_true > 0
    L = y * np.log(p) + (1 - y) * np.log(1.0 - p)
    return -np.mean(L)

def compute_auc(y_true, y_proba):
    """
    computes AUC for a binary classifier quickly
    :param y_true: vector of true classes
    :param y_proba: vector of predicted probabilities
    :return: auc
    """
    n = len(y_true)
    I_pos = y_true > 0
    # if y[i] == 0 for all i or y[i] == 1 for all i, then return AUC = 1
    if I_pos.all() or np.logical_not(I_pos).all():
        return float('nan')

    I_pos = I_pos[np.argsort(y_proba)]
    false_positive_count = np.cumsum(1 - I_pos)
    n_false = false_positive_count[-1]
    auc = np.multiply(I_pos, false_positive_count).sum() / (
            n_false * (n - n_false))
    return auc

def compute_ece(y_true, y_proba, n_bins=10):
    """
    computes expected calibration error for a binary classifier
    :param y_true: vector of true classes
    :param y_proba: vector of predicted probabilities
    :param n_bins: 10
    :return:ece
    """

    # pre-process to improve binning
    sort_idx = np.argsort(y_proba)
    y_pred = y_proba[sort_idx]
    y_true = y_true[sort_idx] > 0
    n = len(y_true)

    acc = np.zeros(n_bins)
    conf = np.zeros(n_bins)
    counts = np.zeros(n_bins)

    for k in range(n_bins):
        left_idx = np.searchsorted(y_pred, k / n_bins, side='left')
        right_idx = np.searchsorted(y_pred, (k + 1) / n_bins, side='right')
        if left_idx < right_idx:
            bin_idx = np.arange(left_idx, right_idx)
```

```
            counts[k] = len(bin_idx)
            conf[k] = np.mean(y_pred[bin_idx])
            acc[k] = np.mean(y_true[bin_idx])

    ece = np.sum(counts * np.abs(acc - conf)) / n
    return ece

def plot_roc_curve(y_true, y_pred, ax, type = 'full'):
    """
    :param y_true:
    :param y_pred:
    :param ax:
    :return:
    """
    assert len(y_true) == len(y_pred)

    if type == 'full':
        plot_args = {'color': "black"}
    elif type == 'fold':
        plot_args = {'color': "grey"}

    # call sklearn
    return RocCurveDisplay.from_predictions(y_true, y_pred, ax = ax, **plot_args)

def plot_reliability_diagram(y_true, y_pred, ax, type = 'full', n_bins = 10):
    """
    :param y_true:
    :param y_pred:
    :param ax:
    :return:
    """
    assert len(y_true) == len(y_pred)

    if type == 'full':
        plot_args = {'color': "black"}
    elif type == 'fold':
        plot_args = {'color': "grey"}

    # call sklearn
    return CalibrationDisplay.from_predictions(y_true, y_pred, strategy = 'uniform', ax = ax, n_bins = n_bins, **plot_args)
```

```
Requirement already satisfied: cvxpy in c:\users\issac\miniforge3\envs\dsc80\lib\site-packages (1.7.3)
Requirement already satisfied: osqp>=1.0.0 in c:\users\issac\miniforge3\envs\dsc80\lib\site-packages (from cvxpy) (1.0.5)
Requirement already satisfied: clarabel>=0.5.0 in c:\users\issac\miniforge3\envs\dsc80\lib\site-packages (from cvxpy) (0.11.1)
Requirement already satisfied: scs>=3.2.4.post1 in c:\users\issac\miniforge3\envs\dsc80\lib\site-packages (from cvxpy) (3.2.9)
Requirement already satisfied: numpy>=1.22.4 in c:\users\issac\miniforge3\envs\dsc80\lib\site-packages (from cvxpy) (2.1.1)
Requirement already satisfied: scipy>=1.13.0 in c:\users\issac\miniforge3\envs\dsc80\lib\site-packages (from cvxpy) (1.14.1)
Requirement already satisfied: cffi in c:\users\issac\miniforge3\envs\dsc80\lib\site-packages (from clarabel>=0.5.0->cvxpy) (1.
17.1)
Requirement already satisfied: jinja2 in c:\users\issac\miniforge3\envs\dsc80\lib\site-packages (from osqp>=1.0.0->cvxpy) (3.1.
4)
Requirement already satisfied: setuptools in c:\users\issac\miniforge3\envs\dsc80\lib\site-packages (from osqp>=1.0.0->cvxpy)
(75.1.0)
Requirement already satisfied: joblib in c:\users\issac\miniforge3\envs\dsc80\lib\site-packages (from osqp>=1.0.0->cvxpy) (1.4.
2)
Requirement already satisfied: pycparser in c:\users\issac\miniforge3\envs\dsc80\lib\site-packages (from cffi->clarabel>=0.5.0-
>cvxpy) (2.22)
Requirement already satisfied: MarkupSafe>=2.0 in c:\users\issac\miniforge3\envs\dsc80\lib\site-packages (from jinja2->osqp>=1.
0.0->cvxpy) (2.1.5)
Note: you may need to restart the kernel to use updated packages.
```

Part (a)

Fit a vanilla logistic regression model for this dataset.

- i. Plot an ROC curve and the reliability diagram for your model. Each plot should include 2 lines: (1) to show the performance of the model on the entire training dataset; (1) to show the mean performance across the 5 folds.
- ii. Report the values of the training loss, AUC, and ECE for your model. Compare them to the values mean 5-CV estimates of the loss, AUC, ECE? Given your results, state if we can use the model for ranking and risk estimation on unseen data.

In [19]:
```
### load dataset
df = pd.read_csv('./adult_processed.csv')
variable_names = df.columns.tolist()[1:]
outcome_name = df.columns.tolist()[0]
y = df[outcome_name].values
X = df[variable_names].values
```

In [20]:
```
### generate folds first, so we use the same for each trial
kf = StratifiedKFold(n_splits = 5, shuffle=True, random_state=1337)
folds = np.repeat(np.nan, X.shape[0])
for k, cv_indices in enumerate(kf.split(X, y)):
    test_indices = cv_indices[1]
    folds[test_indices] = k + 1
folds = folds.astype(int)
```

```
#### fit clf for L1 regression
clf = L1LogisticRegression(C = 0.0)
clf.fit(X, y)
y_proba = clf.predict_proba(X)

# stats for model trained on all data
train_loss = compute_log_loss(y, y_proba)
train_auc = compute_auc(y, y_proba)
train_ece = compute_ece(y, y_proba)
print(r"""\begin{tabular}{lccc}
\hline
 & Logistic Loss & AUC & ECE \\
\hline
Full Training & %.4f & %.4f & %.4f \\
""" % (train_loss, train_auc, train_ece))
```

```
\begin{tabular}{lccc}
\hline
 & Logistic Loss & AUC & ECE \\
\hline
Full Training & 0.3630 & 0.8757 & 0.0102 \\
```

In [21]:

```
# roc/cal plots for models fit on all the data
fig, ax = plt.subplots(1, 2, figsize = (12, 12))
ax[0].set_aspect('equal', adjustable='box') # roc curve
ax[1].set_aspect('equal', adjustable='box') # reliability diagram

# Your Code Here
plot_roc_curve(y, y_proba, ax[0])
plot_reliability_diagram(y, y_proba, ax[1])
cv_losses = []
cv_aucs = []
cv_eces = []

for k in range(1, max(folds)+1):

    clf = L1LogisticRegression(C = 0.0)
    test_idx = np.isin(folds, k)
    train_idx = ~test_idx
    clf.fit(X[train_idx], y[train_idx])
    y_proba = clf.predict_proba(X[test_idx])

    # Your Code Here
    plot_roc_curve(y[test_idx], y_proba, ax[0], type='fold')
    plot_reliability_diagram(y[test_idx], y_proba, ax[1], type='fold')
    # Compute loss, AUC, ECE
    cv_losses.append(compute_log_loss(y[test_idx], y_proba))
    cv_aucs.append(compute_auc(y[test_idx], y_proba))
    cv_eces.append(compute_ece(y[test_idx], y_proba))

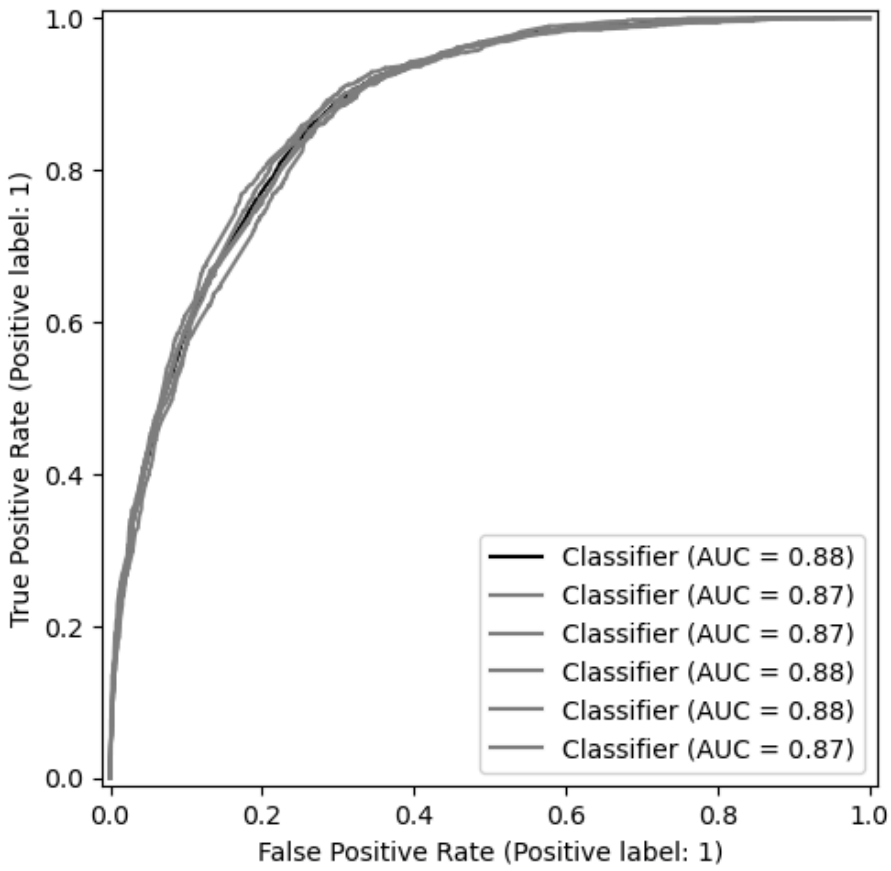
ax[1].set_xlabel("Predicted Probability")
ax[1].set_ylabel("Observed Probability")
plt.show()

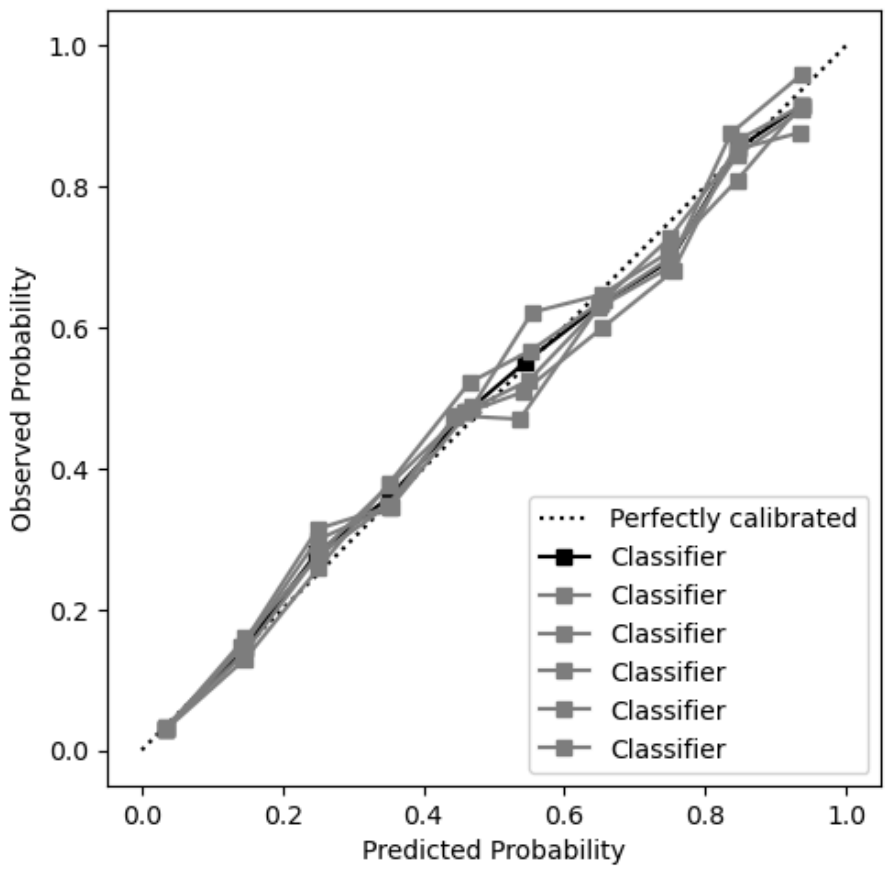
# Compute mean across folds
cv_loss_mean = np.mean(cv_losses)
cv_auc_mean = np.mean(cv_aucs)
cv_ece_mean = np.mean(cv_eces)

# Rows for each fold
for i in range(5):
    print("Fold %d & %.4f & %.4f & %.4f \\\\" % (i+1, cv_losses[i], cv_aucs[i], cv_eces[i]))

# Row for mean
print("Mean (5-Fold CV) & %.4f & %.4f & %.4f \\\\" % (cv_loss_mean, cv_auc_mean, cv_ece_mean))
print(r"\hline")
print(r"\end{tabular}")
```

```
c:\Users\issac\miniforge3\envs\dsc80\Lib\site-packages\cvxpy\problems\problem.py:1539: UserWarning: Solution may be inaccurate. Try another solver, adjusting the solver settings, or solve with verbose=True for more information.
  warnings.warn(
```






```
Fold 1 & 0.3642 & 0.8744 & 0.0148 \\
Fold 2 & 0.3661 & 0.8734 & 0.0125 \\
Fold 3 & 0.3593 & 0.8778 & 0.0141 \\
Fold 4 & 0.3577 & 0.8815 & 0.0144 \\
Fold 5 & 0.3725 & 0.8676 & 0.0190 \\
Mean (5-Fold CV) & 0.3639 & 0.8749 & 0.0150 \\
\hline
\end{tabular}
```

Part (b)

Fit a L_1-penalized logistic regression model by adapting your implementation from Problem Q4. Fit models for L_1-penalties in the set $\lambda \in \{0.001, 0.002, 0.005, 0.01, 0.02, 0.05\}$. Instead of tuning the L_1-penalty to maximize predictive accuracy, let us tune it to induce sparsity by returning the best possible model with a model size of at most 5. Here, *model size* refers to number of non-zero coefficients in your model excluding the intercept term. *Note: you may run into a numerical issue where **cvxpy** returns small coefficients with very small values, rather than values set to zero. To avoid miscalculating model size in this case, you should manually set any coefficient $|w_j| < 10^{-6}$ to 0.*

- i. Plot an ROC curve and the reliability diagram for your model. Again each plot should include 2 lines: (1) to show the performance of the model on the entire training dataset; (1) to show the mean performance across five folds.
- ii. How does L_1-regularization affect calibration? Explain why you observe this effect?
- iii. What is its training loss, AUC, and ECE? What is the 5-CV loss, AUC, and ECE? Can you rely on 5-CV to estimate the performance of your model on unseen data?

In [28]:
```
##### Fit Classifiers for L1 Logistic Regression #####

loss_means, models = [], []
for penalty in tqdm([0.001, 0.002, 0.005, 0.01, 0.02, 0.05]):

    # train full model
    clf = L1LogisticRegression(C = penalty)
    clf.fit(X, y)
    clf.w.value[np.abs(clf.w.value) < 1e-6] = 0
    models.append(clf)

    # Compute stats
    y_proba = clf.predict_proba(X)
    train_loss = compute_log_loss(y, y_proba)
    train_auc = compute_auc(y, y_proba)
    train_ece = compute_ece(y, y_proba)

    cv_losses = []

    # train fold models
    for k in range(1, max(folds) + 1):

        clf = L1LogisticRegression(C = penalty)
        test_idx = np.isin(folds, k)
        train_idx = ~test_idx
        clf.fit(X[train_idx], y[train_idx])
        clf.w.value[np.abs(clf.w.value) < 1e-6] = 0

        # Compute stats
        y_proba = clf.predict_proba(X[test_idx])
        cv_losses.append(compute_log_loss(y[test_idx], y_proba))
```

```
    cv_loss_mean = np.mean(cv_losses)
    loss_means.append(cv_loss_mean)
    cv_auc_mean = np.mean(cv_aucs)
    cv_ece_mean = np.mean(cv_eces)

# Pick the best model
valid_models = [(m, lm) for m, lm in zip(models, loss_means)
                if np.sum(np.abs(m.w.value) > 1e-6) <= 5]
best_model, best_loss = min(valid_models, key=lambda x: x[1])
```

```
 17%|██         | 1/6 [00:29<02:25, 29.16s/it]c:\Users\issac\miniforge3\envs\dsc80\Lib\site-packages\cvxpy\problems\problem.py:
1539: UserWarning: Solution may be inaccurate. Try another solver, adjusting the solver settings, or solve with verbose=True fo
r more information.
  warnings.warn(
100%|██████████| 6/6 [02:38<00:00, 26.34s/it]
```

In [27]:
```
#plot for your chosen model
fig, ax = plt.subplots(1, 2, figsize = (12, 12))
ax[0].set_aspect('equal', adjustable='box') # roc curve
ax[1].set_aspect('equal', adjustable='box') # reliability diagram

# Your Code Here
y_proba = best_model.predict_proba(X)
plot_roc_curve(y, y_proba, ax[0])
plot_reliability_diagram(y, y_proba, ax[1])

losses, aucs, eces = [], [], []

for k in range(1, max(folds)+1):
    test_idx = np.isin(folds, k)
    train_idx = ~test_idx
    y_proba_fold = best_model.predict_proba(X[test_idx])
    plot_roc_curve(y[test_idx], y_proba_fold, ax[0], type='fold')
    plot_reliability_diagram(y[test_idx], y_proba_fold, ax[1], type='fold')

    # Compute Loss, AUC, ECE
    losses.append(compute_log_loss(y[test_idx], y_proba_fold))
    aucs.append(compute_auc(y[test_idx], y_proba_fold))
    eces.append(compute_ece(y[test_idx], y_proba_fold))

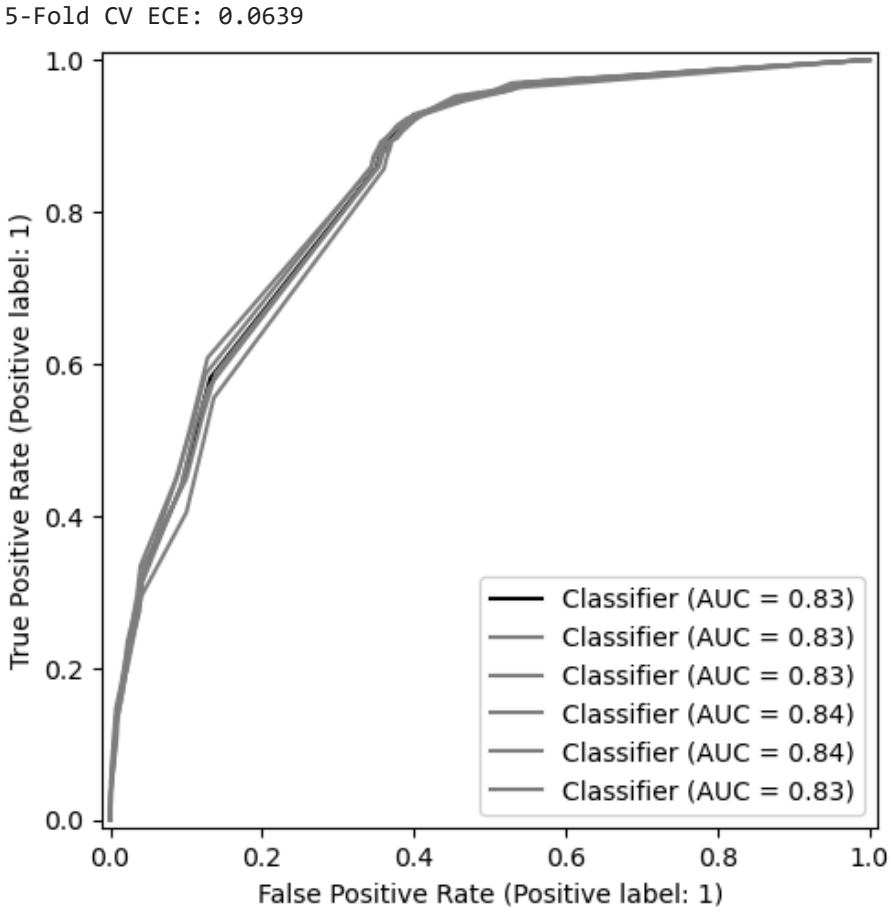
ax[1].set_xlabel("Predicted Probability")
ax[1].set_ylabel("Observed Probability")

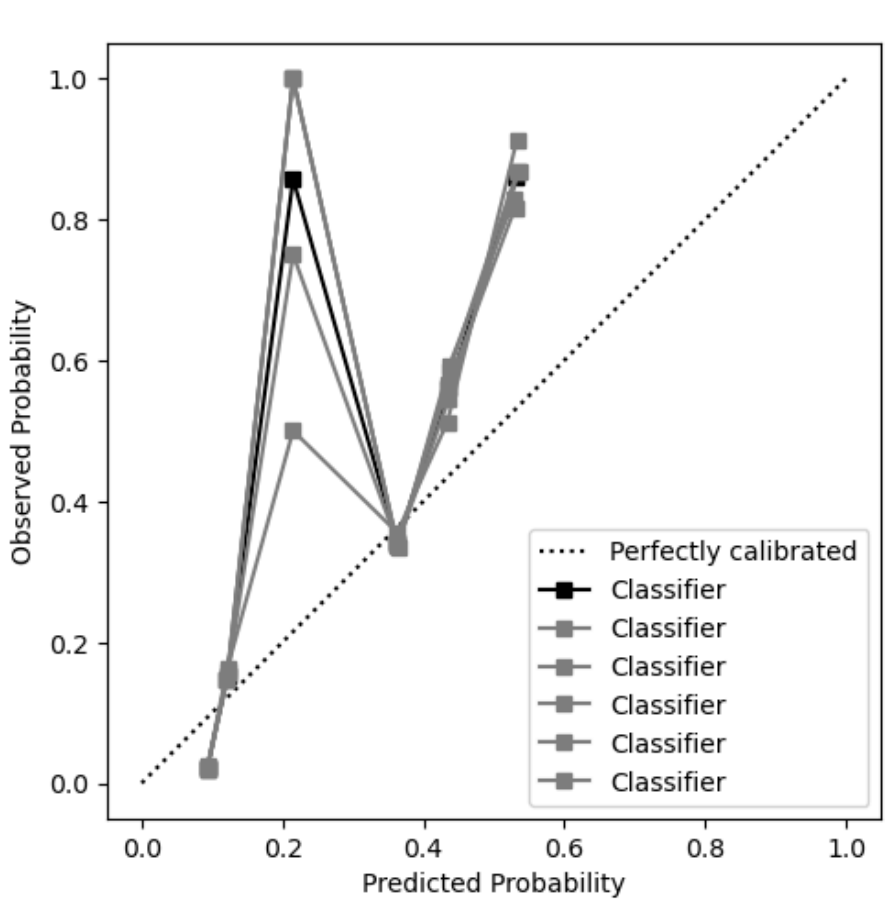
print("Log Loss: %.4f" % compute_log_loss(y, y_proba))
print("AUC: %.4f" % compute_auc(y, y_proba))
print("ECE: %.4f" % compute_ece(y, y_proba))

print("5-Fold CV Log Loss: %.4f" % np.mean(losses))
print("5-Fold CV AUC: %.4f" % np.mean(aucs))
print("5-Fold CV ECE: %.4f" % np.mean(eces))

plt.show()
```

```
Log Loss: 0.4353
AUC: 0.8346
ECE: 0.0639
5-Fold CV Log Loss: 0.4353
5-Fold CV AUC: 0.8354
5-Fold CV ECE: 0.0639
```





Part (c)

Let's see if we can improve the probability predictions from your ``sparse" model by Platt Scaling. Fit a vanilla logistic regression model to post-process the probability predictions from your model in part (b) via Platt scaling.

- i. Plot a reliability diagram to show the probabilities of the model trained on the entire training dataset after Platt Scaling. How do the probability predictions compare to the values you obtained in Part (b)?
- ii. How does Platt Scaling affects ECE? What about AUC? Explain your observations.

In [25]:
```
###### Question 3: Platt Scaling ####

# Your Code Here for Platt Scaling

fig, ax = plt.subplots(1, 2, figsize = (24, 12))
ax[0].set_aspect('equal', adjustable='box')
ax[1].set_aspect('equal', adjustable='box')

# Your Code Here
y_proba = best_model.predict_proba(X)
plot_reliability_diagram(y, y_proba, ax[0])
print("Log Loss: %.4f" % compute_log_loss(y, y_proba))
print("AUC: %.4f" % compute_auc(y, y_proba))
print("ECE: %.4f" % compute_ece(y, y_proba))

platt = L1LogisticRegression(C=0)
X_proba = np.column_stack((np.ones(y_proba.shape[0]), y_proba))
platt.fit(X_proba, y)
y_proba_proba = platt.predict_proba(X_proba)
plot_reliability_diagram(y, y_proba_proba, ax[1])
print("Log Loss: %.4f" % compute_log_loss(y, y_proba_proba))
print("AUC: %.4f" % compute_auc(y, y_proba_proba))
print("ECE: %.4f" % compute_ece(y, y_proba_proba))

ax[0].set_xlabel("Predicted Probability")
ax[0].set_ylabel("Observed Probability")
ax[1].set_xlabel("Predicted Probability")
ax[1].set_ylabel("Observed Probability")
plt.show()
```

```
Log Loss: 0.4353
AUC: 0.8346
ECE: 0.0639
Log Loss: 0.4188
AUC: 0.8346
ECE: 0.0343
```

